

Taller API REST con Spring Boot

Este taller describe, paso a paso, cómo crear un **servicio REST con Spring Boot**, buenas prácticas de arquitectura y documentación, así como incorpora el uso de **DTO (Data Transfer Objects)** para desacoplar la capa de persistencia de la capa web.

Objetivo del taller

Desarrollar una API REST para gestionar personas de una empresa utilizando Spring Boot. Al finalizar, la API permitirá **crear, consultar, modificar y eliminar personas** en una base de datos MYSQL a la que accederemos a través de JPA (**Java Persistence API**), aplicando validaciones y buenas prácticas como DTOs.

Si queréis más información, a continuación, se adjunta una guía para crear un rest service con Spring, paso a paso: <https://spring.io/guides/tutorials/rest> y también una guía de como acceder a los datos usando JPA <https://spring.io/guides/gs/accessing-data-jpa> y <https://spring.io/guides/gs/accessing-data-rest> o bien si nos conectamos a una base de datos mongoDB <https://www.mongodb.com/resources/products/compatibilities/spring-boot> y <https://spring.io/guides/gs/accessing-mongodb-data-rest>

La arquitectura típica en Spring Boot es:

Controller → Service → Repository → Base de datos

- **Controller:** solo gestiona HTTP (GET, POST, PUT, DELETE), no tiene lógica de negocio, no accede a la base de datos
- **Service:** lógica de negocio
- **Repository:** acceso a datos
- **(DTOs** opcional pero recomendado)

Preparación del entorno

Crear en MYSQL una base de datos llamada **personas**, y una tabla llamada **PERSONA** con los siguientes campos y datos de prueba.

```
CREATE DATABASE personas;
USE personas;

CREATE TABLE PERSONA (
    ID_PERSONA INT NOT NULL AUTO_INCREMENT COMMENT 'Identificador de la persona',
    NOMBRE varchar(45) NOT NULL COMMENT 'Nombre de la persona',
    APELLIDOS varchar(200) NOT NULL COMMENT 'Apellidos de la persona',
    DOMICILIO varchar(100) NULL COMMENT 'Domicilio de la persona',
    EMAIL varchar(100) NULL COMMENT 'Email de la persona',
    CONSTRAINT PK_PERSONA PRIMARY KEY (ID_PERSONA)
);

INSERT INTO PERSONA (ID_PERSONA, NOMBRE, APELLIDOS, DOMICILIO, EMAIL)
VALUES (1, 'Yolanda', 'Moreno', 'C/Alarcos Ciudad Real', 'yolanda@gmail.com');
```

NOTA: Si no tenemos montado un servidor MYSQL, podéis usar XAMP (<https://www.apachefriends.org/es/download.html>) o crear un servidor en vuestro docker, para lo que tenéis que ejecutar en cmd la siguiente orden, la cual descargará la última versión de la imagen mysql del repositorio oficial Docker Hub, y creará el contenedor con nombre **mysql** automáticamente, indicando **root** como contraseña del usuario root.

```
docker run -d --name mysql -e MYSQL_ROOT_PASSWORD=root -p 3306:3306 mysql:latest
```

Como cliente utilizaremos **MySQL Workbench**, os lo podéis descargar de:

<https://www.mysql.com/products/workbench>

Si se va a usar como base de datos MongoDB, puedes crear el contenedor en Docker con:

```
docker run --name mongodb -p 27017:27017 -d mongodb/mongodb-community-server:latest
```

Como cliente puedes usar **MongoDB Compass**, os lo podéis descargar de:

<https://www.mongodb.com/try/download/compass>

Si vais a usar como base de datos Oracle, puedes crear el contenedor en Docker con:

```
docker run -d --name oracle -p 1521:1521 -p 5500:5500 -e ORACLE_PASSWORD=Ora1234 -e ORACLE_CHARACTERSET=AL32UTF8 gvenzl/oracle-free:slim
```

Creación del proyecto

1) Vamos a usar **Spring Initializr** <https://start.spring.io/> , para crear la estructura inicial del proyecto con las siguientes **dependencias**:

- **Spring Web** para poder construir el servicio REST
- **Spring Data JPA** para suministrar acceso a los datos (Java Persistence API para bases de datos relacionales)
- **Spring Data MongoDB** para suministrar acceso a los datos en mongoDB
- **MySQL Driver** driver para poder conectarnos con una base de datos MySQL
- **Oracle Driver** driver para poder conectarnos con una base de datos Oracle
- **MongoDB** es una base de datos de documentos NoSQL de código abierto que utiliza un esquema similar a JSON en lugar de los datos relacionales tradicionales basados en tablas
- **Validation** para validar los datos de entrada / salida (Spring Validation)
- **SpringDOC** para automatizar la generación de documentación de APIs REST en aplicaciones Spring Boot, así como probar las operaciones

Una vez añadidas todas las dependencias, descargamos la estructura del proyecto pulsando en el botón **GENERATE**.



Project
☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Maven**

Language
☒ **Java** ☐ Kotlin ☐ Groovy

Spring Boot
☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (M4) ☐ 4.0.6 (SNAPSHOT) ☒ **4.0.5**
☐ 3.5.14 (SNAPSHOT) ☐ 3.5.13

Project Metadata
Group
Artifact
Package name
Packaging ☒ **Jar** ☐ War
Configuration ☒ **Properties** ☐ YAML
Java ☐ 26 ☐ 25 ☒ **21** ☐ 17

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Web **WEB**
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA **SQL**
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver **SQL**
MySQL JDBC driver.

Validation **IO**
Bean Validation with Hibernate validator.

SpringDoc OpenAPI **WEB**
Add OpenAPI / Swagger documentation to web-based Spring applications.

- 2) Con el proyecto descargado, descomprimir el fichero demo.zip dentro de un workspace (directorio de espacio de trabajo) nuevo.

NOTA: Si tu versión de JDK es distinta a la indicada en el pom.xml, edita el fichero y pon la versión que estés utilizando.

Importar el proyecto demo en vuestro IDE como Proyecto existente de Maven.

En este momento las dependencias que vienen en el fichero pom.xml, comienzan a descargarse a nuestro repositorio local, por lo que es importante tener conexión a Internet.

- 3) Revisamos el fichero de configuración: **application.properties** ¹

```
spring.application.name=demo
```

```
#Puerto del servidor Tomcat  
server.port=8085
```

```
# swagger-ui custom path  
springdoc.swagger-ui.path=/swagger-ui.html
```

```
# Database Properties (MYSQL)  
spring.datasource.url=jdbc:mysql://localhost:3306/personas?serverTimezone=UTC  
spring.datasource.username=root  
spring.datasource.password=root
```

```
# Database Properties (ORACLE)  
spring.datasource.url=jdbc:Oracle:thin://localhost:1521/freepdb1  
spring.datasource.username=persona  
spring.datasource.password=persona
```

```
# Database Properties (MONGODB)  
spring.data.mongodb.host=localhost  
spring.data.mongodb.port=27017  
spring.data.mongodb.database=mi_basedatos  
spring.data.mongodb.username=usuario  
spring.data.mongodb.password=password
```

```
#JPA (BD relacionales)  
spring.jpa.show-sql=true  
spring.jpa.hibernate.naming.physicalstrategy=org.hibernate.boot.model.naming.  
PhysicalNamingStrategyStandardImpl  
spring.jpa.hibernate.naming.implicit-  
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
```

¹ <https://docs.spring.io/spring-boot/docs/1.1.0.M1/reference/html/howto-database-initialization.html>

4) Generar la clase de persistencia de la tabla **Persona**, usando entidades JPA:

- `@Entity` es una anotación JPA que identifica a una clase Java que representa una tabla en mi base de datos.
- `@Table` se utiliza para **especificar los detalles de la tabla de la base de datos** (nombre, esquema) a la que se mapea una entidad
- `Nombre, apellidos, domicilio, email` son los atributos de nuestro objeto `Persona`
- `@id` indica que el campo indicado es la clave principal y el proveedor JPA lo va completar automáticamente.
- `@GeneratedValue` en JPA se utiliza para configurar la **generación automática de claves primarias** en las entidades de base de datos
- El constructor vacío, existe solo para fines de JPA. No lo usas directamente, por lo que debes crearlo como protegido
- Se debe crear un constructor personalizado, pero sin el `id` y los getters y setters de los diferentes atributos de la entidad.

```
package persistence;
```

```
import jakarta.persistence.Column;  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.Table;
```

```
@Entity  
@Table(name="PERSONA", schema="personas")  
public class Persona implements java.io.Serializable{  
  
    @Id  
    @Column(name="ID_PERSONA")  
    @GeneratedValue(strategy= GenerationType.IDENTITY)  
    private Long idPersona;  
  
    @Column(name="NOMBRE")  
    private String nombre;  
  
    @Column(name="APELLIDOS")  
    private String apellidos;  
  
    @Column(name="DOMICILIO")  
    private String domicilio;  
  
    @Column(name="EMAIL")  
    private String email;
```

```
    // Crear constructor vacio  
    // Crear constructor con nombre, apellidos, domicilio y email  
    // Crear Getters y setters  
}
```

Si se utiliza una bbdd Oracle donde la tabla usa **secuencias** para el ID, la **clase Entity** se define usando `@SequenceGenerator`

```
@Id
@GeneratedValue(
    strategy = GenerationType.SEQUENCE,
    generator = "usuario_seq"
)
@SequenceGenerator(
    name = "usuario_seq",
    sequenceName = "SEQ_PERSONA",
    allocationSize = 1 //para que los valores aumenten de 1 en 1
)
@Column(name = "ID_PERSONA")
private Long idPersona;
```

Si se utiliza como bbdd MongoDB, se usará la anotación `@Document(collection = "personas")`. Importante tener en cuenta que MongoDB genera IDs de tipo String

```
package persistence;
```

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
```

```
@Document(collection="personas")
public class Persona implements java.io.Serializable{
```

```
    @Id
    private String idPersona;
    private String nombre;
    private String apellidos;
    private String domicilio;
    private String email;
```

```
    // Crear constructores necesarios
    // Crear Getters y setters
}
```

Otras anotaciones de utilidad pueden ser:

- `@Transient` permite ignorar un atributo, de manera que no se persista la información del objeto en la base de datos. En una base de datos relacional, importamos la anotación para JPA de `jakarta.persistence.Transient`, pero para una base de datos documental como mongodb usaremos la anotación desde `org.springframework.data.annotation.Transient`.

Vamos a usar DTO (Data Transfer Object) para **transferir datos entre capas**, evitando exponer directamente las entidades JPA y desacoplar la parte del API de la base de datos. De tal forma que en el controlador que crearemos más adelante, solo trabajaremos con DTO y en ningún caso con entidades JPA.

Vamos a definir la clase **PersonaDTO**, indicando los datos que se esperan en la capa cliente.

```
public class PersonaDTO {  
  
    private Long id;  
  
    private String nombre;  
  
    private String apellidos;  
  
    private String domicilio;  
  
    private String email;  
  
    // crear getters y setters  
    // crear constructores necesarios  
  
}
```

Podemos validar si los datos de entrada o de salida son correctos usando **Spring Validation**², basado en **Jakarta Validation**³.

Editaremos la clase **PersonaDTO** que hemos hecho, para indicar que validaciones queremos realizar. El estándar **Jakarta Validation** define **anotaciones** que se colocan sobre atributos, métodos y clases para validar datos automáticamente. Tener en cuenta que también se pueden crear validaciones personalizadas.

Algunas validaciones para cadenas de texto muy comunes son:

@NotBlank Debe tener texto no vacío y no nulo.

@NotEmpty No puede estar vacío (""), pero puede contener espacios.

@NotNull No puede ser null.

@Size(min, max) Tamaño mínimo y máximo del String.

@Email Debe tener formato válido de email.

@Pattern(regex) Coincidir con una expresión regular.

Validaciones comunes para campos numéricos:

@Min(valor) Valor mínimo permitido.

@Max(valor) Valor máximo permitido.

@Positive Debe ser mayor que cero.

@PositiveOrZero Mayor o igual que cero.

@Negative Menor que cero.

² <https://docs.spring.io/spring-framework/reference/core/validation/beanvalidation.html>

³ <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/beanvalidation/bean-validation/bean-validation.html>

@NegativeOrZero Menor o igual que cero.

@Digits(integer, fraction) Número con parte entera y decimal limitada.

Validaciones para fechas:

@Past Fecha anterior a hoy.

@PastOrPresent Hoy o antes.

@Future Debe ser futura.

@FutureOrPresent Hoy o después.

Validaciones sobre colecciones o listas:

@NotEmpty Colección con al menos un elemento.

@Size(min, max) Número de elementos permitido.

@Valid Valida objetos dentro de la colección.

```
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class PersonaDTO {

    private Long id;

    @NotBlank
    @Size(max = 45)
    private String nombre;

    @NotBlank
    @Size(max = 200)
    private String apellidos;

    private String domicilio;

    @Email
    private String email;

    // crear getters y setters
}
```

Después de usar las anotaciones de validación en las clases DTO, tenemos que usar la anotación **@Validated** a nivel del servicio o bien si solo se quieren validar algunos métodos, se puede hacer solo en el método que queremos validar. Pero si no se pone esta anotación, no se realizará la validación.

Además, usaremos la anotación **@Valid** en los parámetros o datos devueltos que queremos que se validen.

@Validated y **@Valid** lo usaremos en la clase controlador que veremos más adelante.

Necesitamos realizar una conversión entre la entidad JPA y el DTO, lo cual podemos hacerlo manualmente o usando librerías como **MapStruct**.

Ejemplo manual:

```
public class PersonaMapper {

    public static PersonaDTO toDTO(Persona persona) {
        PersonaDTO dto = new PersonaDTO();
        dto.setId(persona.getIdPersona());
        dto.setNombre(persona.getNombre());
        dto.setApellidos(persona.getApellidos());
        dto.setDomicilio(persona.getDomicilio());
        dto.setEmail(persona.getEmail());
        return dto;
    }

    public static Persona toEntity(PersonaDTO dto) {

        Persona persona = new Persona(
            dto.getNombre(),
            dto.getApellidos(),
            dto.getDomicilio(),
            dto.getEmail()
        );
        //para que funcione la modificacion
        if (null != dto.getId() && (dto.getId() != 0)) {
            persona.setIdPersona(dto.getId());
        }

        return persona;
    }

    public static Persona toEntityCreate(PersonaDTO dto) {

        Persona persona = new Persona(
            dto.getNombre(),
            dto.getApellidos(),
            dto.getDomicilio(),
            dto.getEmail()
        );
        return persona;
    }
}
```

- 5) Como vamos a necesitar una funcionalidad básica CRUD (Create, Read, Update, Delete) sobre la entidad *Persona*, debemos definir la interfaz *PersonaRepository* que extienda de *JpaRepository*, especificando el tipo de dominio como *Persona* y el tipo del id como *Long*, ya que el campo que es PK en *Persona* es de tipo *Long*.

JpaRepository es una interfaz de **Spring Data JPA**⁴ que facilita la gestión de base de datos en aplicaciones Java. Los repositorios JPA de Spring Data son interfaces con métodos que admiten la creación, lectura, búsqueda, modificación y eliminación de registros en un almacén de datos.

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;
import persistence.Persona;

public interface PersonaRepository extends JpaRepository<Persona, Long> {

    //Se pueden definir operaciones personalizadas
    List<Persona> findByNombre(String nombre);

    @Query("SELECT p FROM PERSONA p WHERE p.NOMBRE = :nombre")
    List<Persona> buscarPorNombre(@Param("nombre") String nombre);
}
```

Si se usa MongoDB⁵, tenemos que crear una clase que extienda de *MongoRepository*. Tendremos métodos como *findAll()*, *save()* y *findById()*.

```
package com.example.demo;

import org.springframework.data.mongodb.repository.MongoRepository;
import persistence.Persona;

public interface PersonaRepository extends MongoRepository<Persona, String> {

    //Se pueden definir operaciones personalizadas
    List<Persona> findByNombre(String nombre);

    // ?0 es el primer parametro
    @Query("{ 'nombre' : ?0 }")
    List<Persona> buscarPorNombre(String nombre);
}
```

⁴ <https://spring.io/projects/spring-data-jpa>

⁵ <https://www.mongodb.com/resources/products/compatibilities/spring-boot>

- 6) Para envolver tu repositorio con una capa web, debes recurrir a **Spring MVC**⁶. Solo necesitas crear un controlador Rest (usaremos la anotación `@RestController`) con el cual gestionaremos solicitudes HTTP (GET, POST, etc.) el cual convierte automáticamente las respuestas a formato **JSON** o **XML** sin necesidad de devolver una vista HTML, definiendo métodos a los que se podrá acceder desde la parte WEB.

Con `@RequestMapping`, marcamos la ruta de acceso para el servicio

Las anotaciones `@GetMapping`, `@PostMapping`, `@PutMapping` y `@DeleteMapping`, marcan la ruta de acceso e indican el método de llamada HTTP: **GET, POST, PUT y DELETE**.

- **GET:** Solicita al servidor el recurso especificado en la URL. Las solicitudes que utilizan este método solo deben recuperar datos.
- **POST:** Envía datos al servidor para crear un recurso. Los datos se incluyen en el cuerpo de la solicitud.
- **PUT:** Reemplaza todas las representaciones actuales del recurso de destino con los datos de la solicitud (similar a POST).
- **DELETE:** Elimina el recurso especificado.

```
@CrossOrigin(origins = "*")
@EntityScan(basePackages = {"persistence"})
@RestController
@RequestMapping("/persona")
public class PersonaController {

    @Autowired
    private PersonaRepository repository;

    /**
     * Get all persona list.
     *
     * @return the list of PersonaDTO
     */
    @GetMapping("/find") // GET Method for reading operation
    public List<PersonaDTO> getAllPersons() {

        return repository.findAll()
            .stream()
            .map(PersonaMapper::toDTO)
            .toList();
    }
}
```

La anotación `@CrossOrigin` en Spring Framework sirve para permitir que un servidor acepte peticiones HTTP de un cliente alojado en un dominio, puerto o protocolo diferente. Evita errores de seguridad en el navegador ("Same-Origin Policy") al permitir el intercambio de recursos entre orígenes cruzados.

⁶ <https://docs.spring.io/spring-framework/reference/web/webmvc.html>

La anotación `@EntityScan` es una funcionalidad, principalmente utilizada en frameworks de Java como Spring Boot (con JPA/Hibernate), que sirve para decirle a la aplicación dónde buscar las clases que representan tablas de una base de datos (llamadas "Entidades").

La anotación `@Autowired` de Spring Framework sirve para **inyectar dependencias automáticamente** en las aplicaciones Java, sin necesidad de crearlos manualmente con `new`.

Podemos añadir más operaciones a nuestra clase *PersonaController*:

- Si queremos que nos devuelva toda la información de una Persona por su ID (SELECT * FROM PERSONA WHERE ID_PERSONA=?", podemos crear el siguiente método.

```
/**
 * Get Persona by id
 * @param personaId
 * @return PersonaDTO
 * @throws PersonaNotFoundException
 */
@GetMapping("/find/{id}") // GET Method for Read operation
public ResponseEntity<PersonaDTO> getPersonById(@PathVariable(value = "id") Long personaId) throws PersonaNotFoundException {

    PersonaDTO persona = repository.findById(personaId)
        .map(PersonaMapper::toDTO)
        .orElseThrow(() -> new PersonaNotFoundException(personaId));

    return ResponseEntity.ok(persona);
}
```

Vemos que usamos **PersonaNotFoundException** para indicar que ocurrió un error a través de una excepción personalizada porque no se encontró ningún registro para el id indicado.

```
public class PersonaNotFoundException extends RuntimeException {

    public PersonaNotFoundException(Long id) {
        super("Could not find persona " + id);
    }

}
```

- Si queremos crear un registro en la tabla PERSONA, podemos crear el siguiente método. Aquí hemos usado la anotación **@Validated** para habilitar la validación a nivel de método, ya que, si no se pone esta anotación, no se realizará la validación. Usaremos la anotación **@Valid** en los parámetros o datos revueltos que queremos que se validen.

¿Qué ocurre si la validación no se cumple? Prueba a crear una persona cuyo email no tenga la @ , vemos que como respuesta **devuelve código de error 400 Bad Request**.

Si miramos el log de Tomcat, vemos el error de validación que se ha producido: **debe ser una dirección de correo electrónico con formato correcto**

```
/**
 * Crea persona entity
 * @param newPersona PersonaDTO
 * @return PersonaDTO
 */
@Validated //habilita la validacion a nivel de metodo
@PostMapping("/create") // POST Method for Create operation
public ResponseEntity<PersonaDTO> createPerson(@Valid @RequestBody PersonaDTO newPersona){
    Persona persona = repository.save(PersonaMapper.toEntityCreate(newPersona));
    return ResponseEntity.ok().body(PersonaMapper.toDTO(persona));
}
```

- Si queremos modificar un registro de la tabla PERSONA, podemos crear el siguiente método.

```
/**
 * Update person response entity
 * @param personDetails
 * @param personaId
 * @return PersonaDTO
 * @throws PersonaNotFoundException
 */
@PutMapping("/update/{id}") // PUT Method for Update operation
public ResponseEntity<PersonaDTO> updatePerson(@RequestBody PersonaDTO personDetails,
    @PathVariable(value = "id") Long personaId) throws PersonaNotFoundException {
    PersonaDTO persona = repository.findById(personaId)
        .map(PersonaMapper::toDTO)
        .orElseThrow(() -> new PersonaNotFoundException(personaId));

    if (null != personDetails.getNombre()) {
        persona.setNombre(personDetails.getNombre());
    }
    if (null != personDetails.getApellidos()) {
        persona.setApellidos(personDetails.getApellidos());
    }
    if (null != personDetails.getDomicilio()) {
        persona.setDomicilio(personDetails.getDomicilio());
    }
    if (null != personDetails.getEmail()) {
        persona.setEmail(personDetails.getEmail());
    }

    final Persona personaModificada = repository.save(PersonaMapper.toEntity(persona));
    return ResponseEntity.ok(PersonaMapper.toDTO(personaModificada));
}
```

- Si queremos eliminar un registro de la tabla PERSONA a partir de su ID, podemos crear el siguiente método.

```
/**
 * Delete person by ID
 * @param personaId
 * @return true if the entity has been deleted, false in other case
 * @throws PersonaNotFoundException
 */
@DeleteMapping("/delete/{id}") // DELETE Method for Delete operation
public boolean deletePerson(@PathVariable(value = "id") Long personaId) throws PersonaNotFoundException {
    PersonaDTO persona = repository.findById(personaId)
        .map(PersonaMapper::toDTO)
        .orElseThrow(() -> new PersonaNotFoundException(personaId));

    repository.delete(PersonaMapper.toEntity(persona));
    return true;
}
```

- 7) Para ejecutar nuestra aplicación usaremos la clase que se nos creó automáticamente usando **spring initializr**. La anotación `@SpringBootApplication` inicia un contenedor de servlets (Tomcat) y arranca nuestro servicio.

```
package com.example.demo;

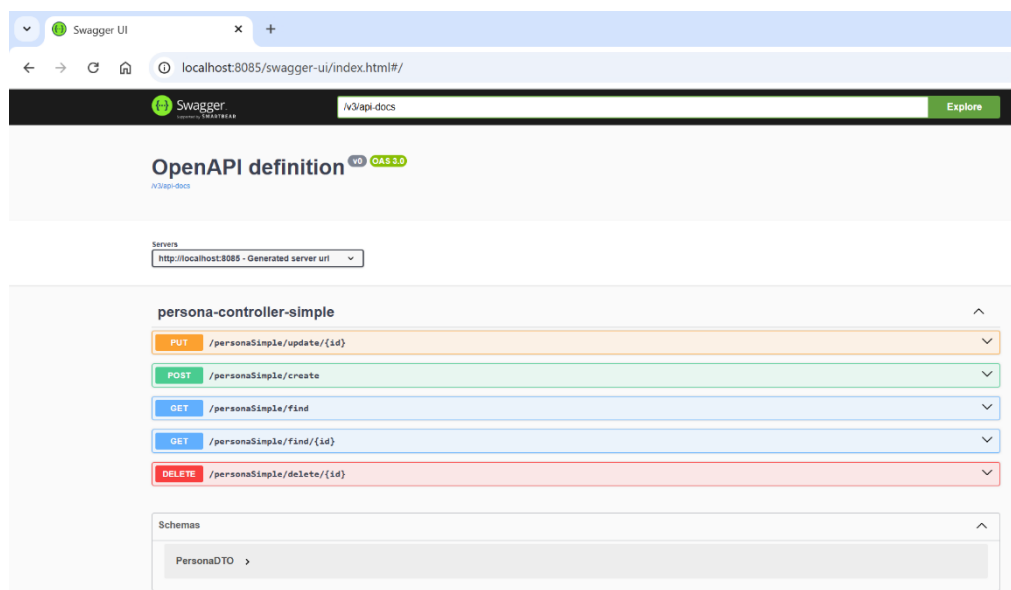
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

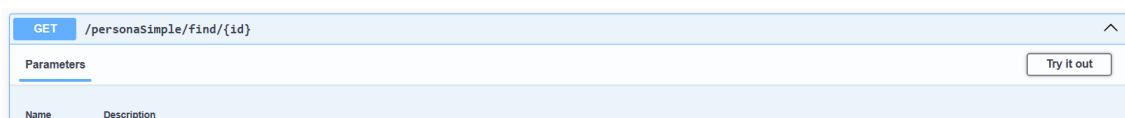
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

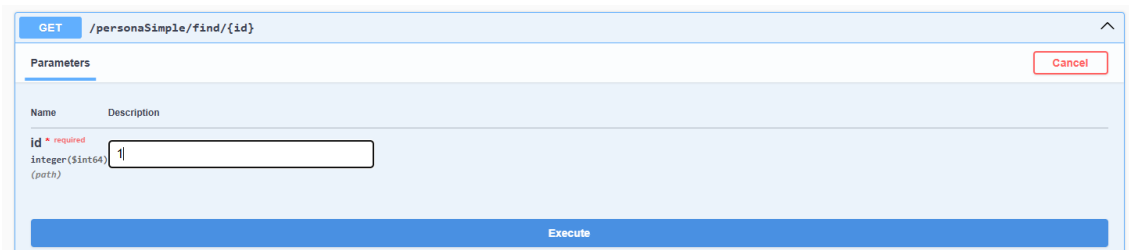
- 8) Para probar el REST service, arrancamos nuestro servicio y accedemos a <http://localhost:8085/swagger-ui.html> desde donde podemos probar las operaciones definidas en el controlador.



Vamos a probar la operación **/persona/find/{id}**, para lo que pulsaremos el botón **TRY IT OUT**



Ponemos el id de un registro que exista en base de datos y pulsamos **Execute**



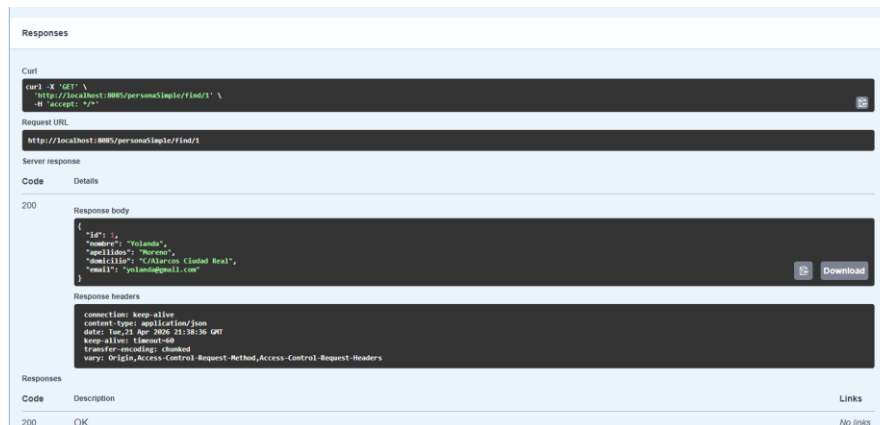
GET /personaSimple/find/{id}

Parameters

Name	Description
id ^{* required} Integer(Integer) (path)	1

Execute

Vemos la respuesta obtenida:



Responses

Curl

```
curl -X GET 'http://localhost:8085/personaSimple/find/1' \
-H 'accept: */*'
```

Request URL

http://localhost:8085/personaSimple/find/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "nombre": "Yolanda", "apellidos": "Moreno", "email": "Yolanda@iesmaestros.com", "email": "yolanda@gmail.com" }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Tue, 21 Apr 2020 21:38:36 GMT keep-alive: 120000 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

Responses

Code	Description	Links
200	OK	No links

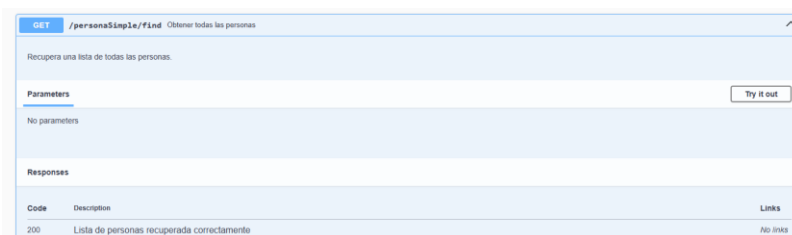
- 9) Podemos documentar el servicio rest que acabamos de crear usando **Springdoc**⁷, el cual está basado en **Swagger**⁸.

Podemos añadir a nuestro código anotaciones para documentar nuestra aplicación:

```
/**
 * Get all persona list.
 *
 * @return the list
 */
@Operation(summary = "Obtener todas las personas", description = "Recupera una lista de todas las personas.")
@ApiResponse(responseCode = "200", description = "Lista de personas recuperada correctamente")
@GetMapping("/find") // GET Method for reading operation
public List<Persona> getAllPersons() {

    return repository.findAll();
}
```

Arrancamos nuestro servicio y accedemos a <http://localhost:8085/swagger-ui.html> y vemos toda la documentación:



GET /personaSimple/find Obtener todas las personas

Recupera una lista de todas las personas.

Parameters

No parameters

Responses

Code	Description	Links
200	Lista de personas recuperada correctamente	No links

⁷ <https://springdoc.org/#getting-started>

⁸ <https://swagger.io/docs/>